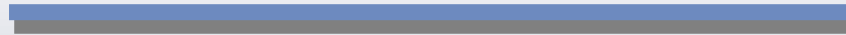


Artificial Neural Networks



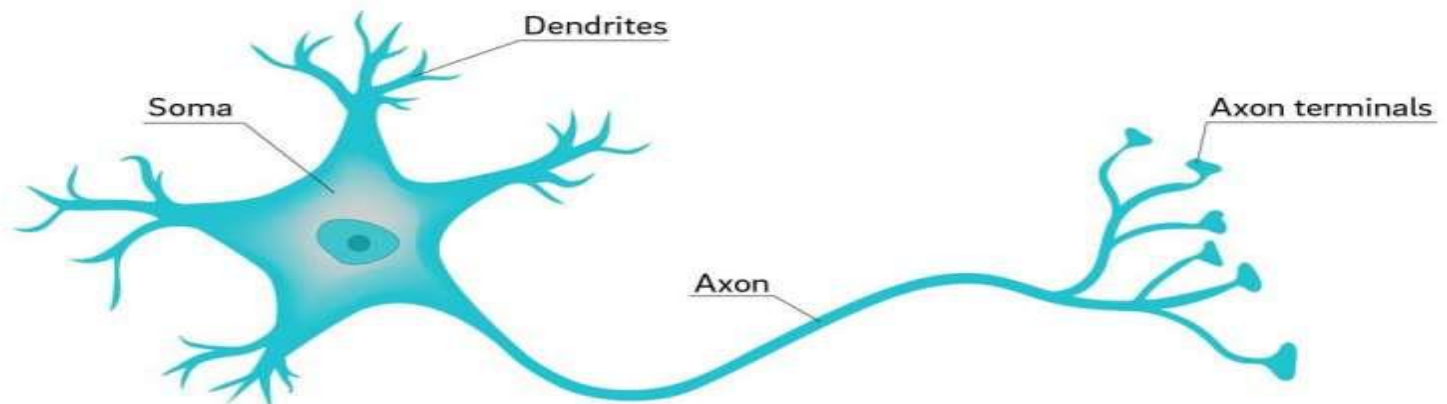
Artificial Neural Networks

- Information processing systems designed with inspiration taken from the nervous system, specifically the brain, and with particular emphasis in problem solving
- A method of computing, based on the interaction of multiple connected processing elements.
- mathematical models for information processing, which are based on the biological models and mechanisms of human brain activities.

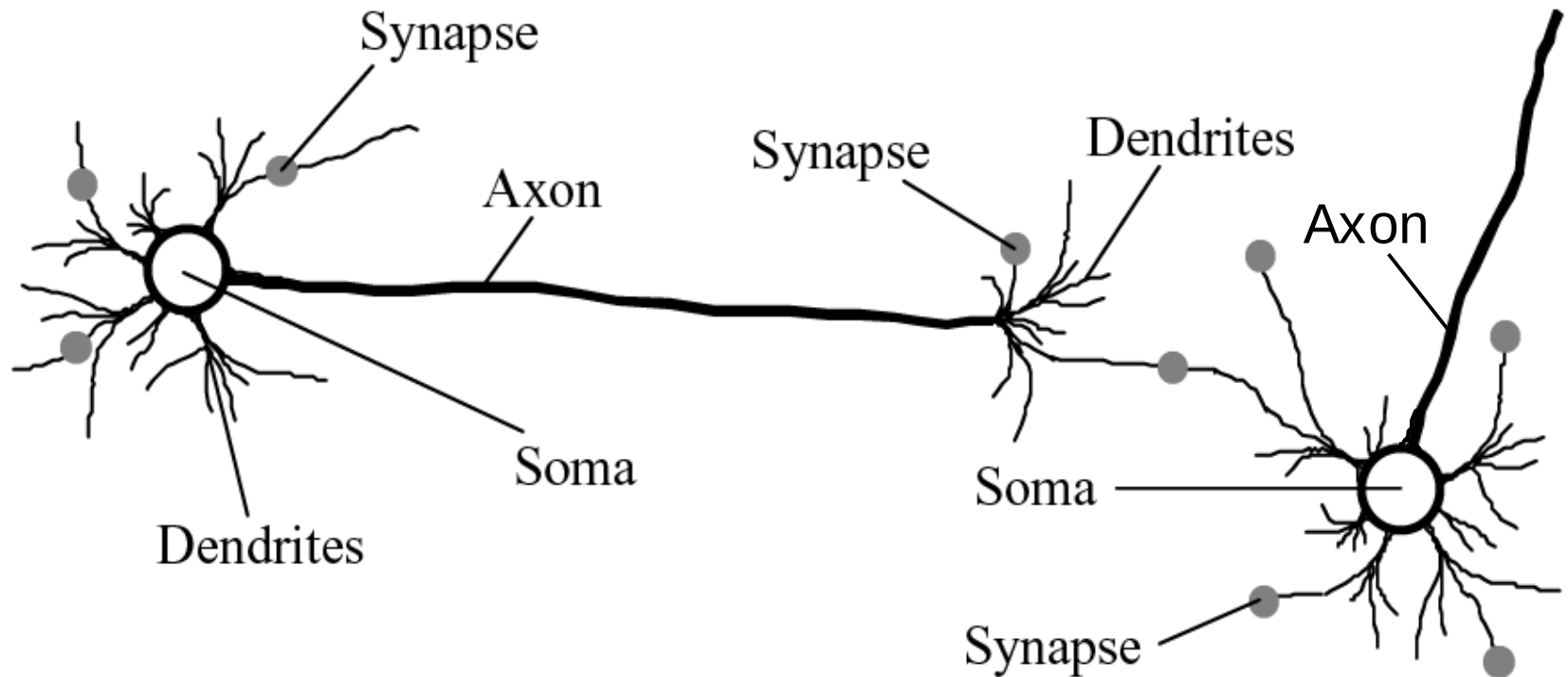
A Neuron

- The basic element of the brain is a natural neuron; similarly, the basic element of every neural network is an **artificial neuron**, or simply **neuron**.
 - A neuron is the **basic building block** for all types of neural networks.
- A neuron consists of a cell body, **soma**, a number of fibers called **dendrites**, and a single long fiber called the **axon**.

Neuron



Biological Neural Network



It is estimated that there is in the order of **10-500 BILLION** neurons, With **60 TRILLION** connections, synapses. The neurons are arranged in approximately **1000 main modules**, each module having about **500 neural networks**.

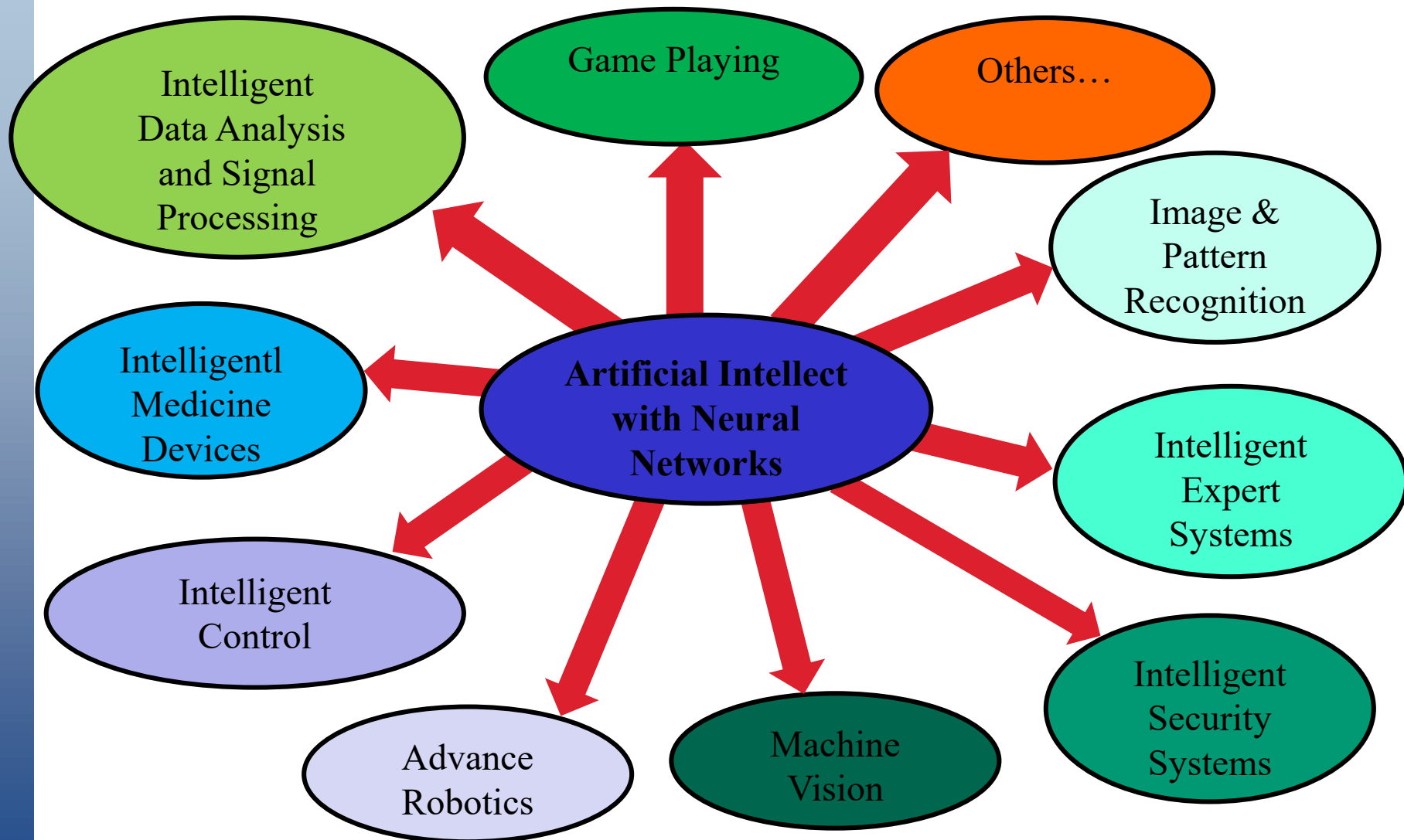
- Our brain can be considered as a highly complex, non-linear and parallel information-processing system.
- Information is stored and processed in a neural network simultaneously throughout the whole network, rather than at specific locations.
- Learning is a fundamental and essential characteristic of biological neural networks. The ease with which they can learn led to attempts to simulate a biological neural network in a computer.

- An artificial neural network consists of a number of very simple processors, also called **neurons**, which are similar to the biological neurons in the brain.
- The neurons are connected by weighted links passing signals from one neuron to another.

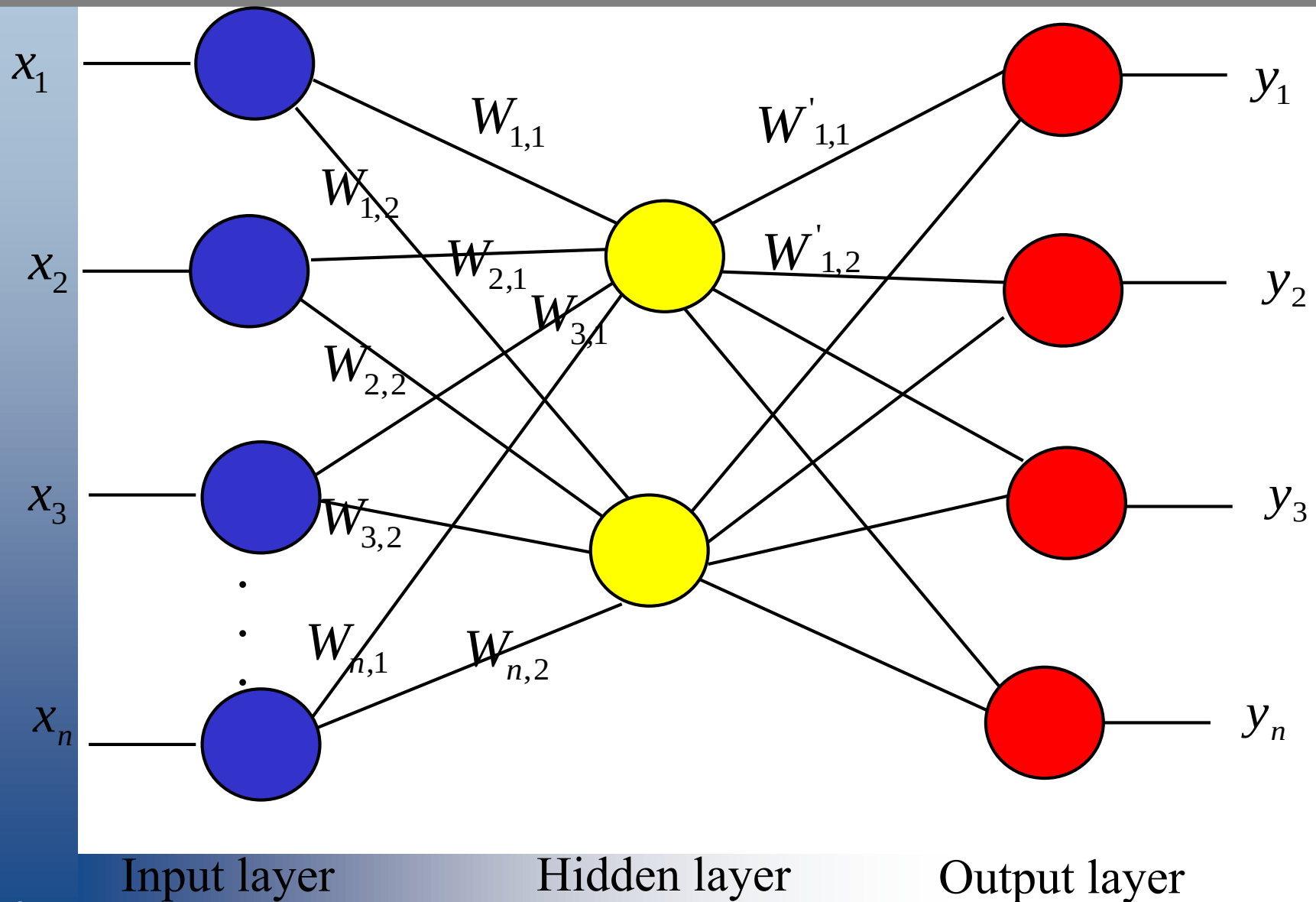
Properties of Nervous Systems

- Parallel, distributed information processing
- High degree of connectivity among basic units
- Connections are modifiable based on experience
- Learning is a constant process, and usually unsupervised
- Learning is based only on local information

Applications of Artificial Neural Networks



Architecture of a typical artificial neural network



Analogy between biological and ANNs

- The biological terms of human nervous systems are introduced for designing the artificial nervous and designing the artificial nervous system/artificial neural network

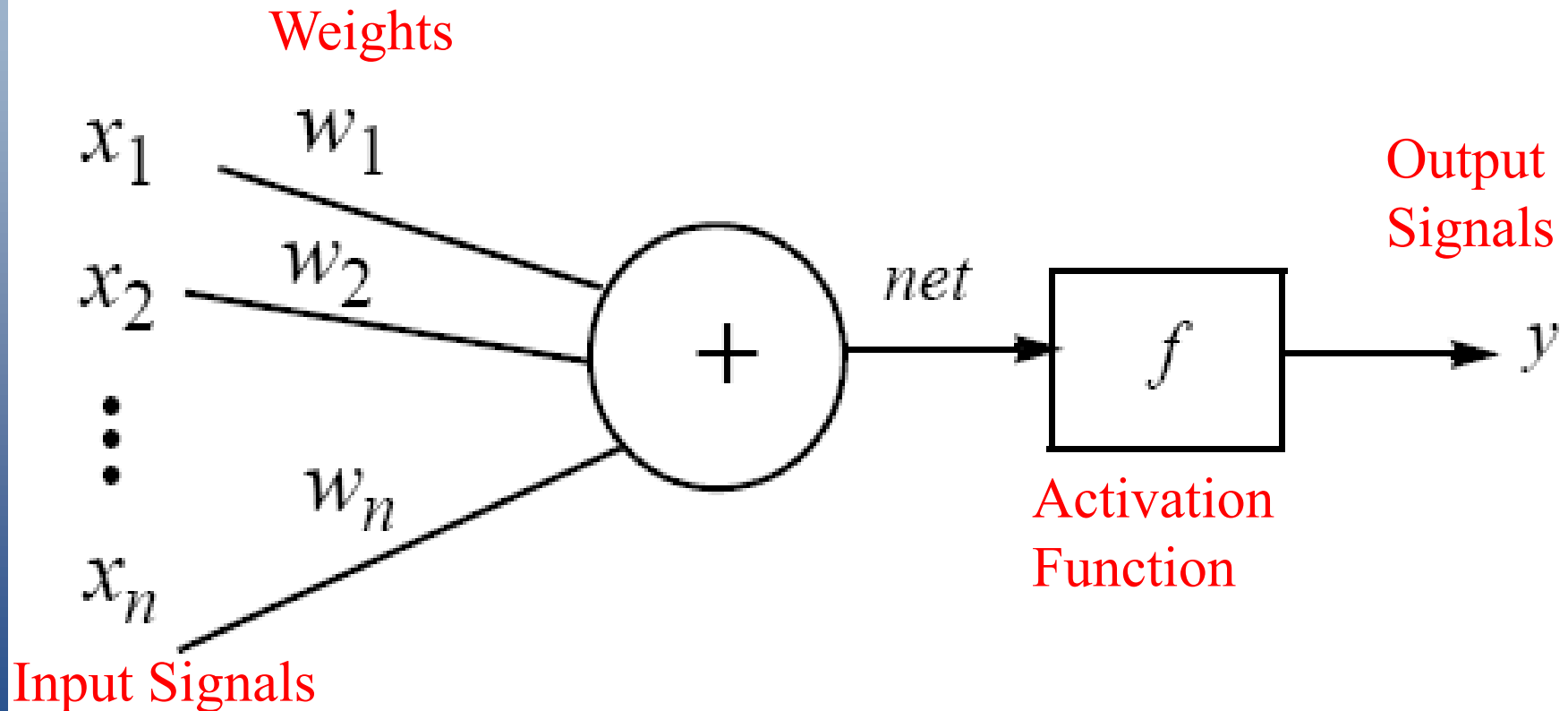
Biological Nervous System	Artificial Terms
Soma	Neuron
Dendrite	Input
Axon	Output
Synapse	Weight

Model of an artificial neuron Terminology

1. $x_1, x_2, \dots, x_n$ are the inputs to the neuron
2. $w_1, w_2, \dots, w_n$ are real-valued parameters called weights
3. $net = w_1 x_1 + w_2 x_2 + \dots + w_n x_n$ is called the weighted sum
4. f : is called the activation function
5. $y = f(net)$ is the output of the neuron

The neuron as a simple computing element

Diagram of a neuron



Calculating the net input

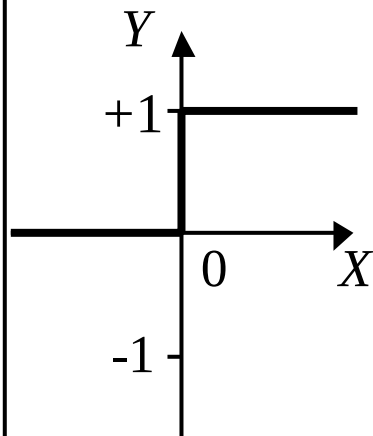
The net input signal to an ANN is usually computed as the weighted sum of all input signals,

$$net = \sum_{i=0}^{n-1} x_i w_i$$

Artificial neurons that compute the net input signal as the weighted sum of input signals are referred as **summation units (SU)**.

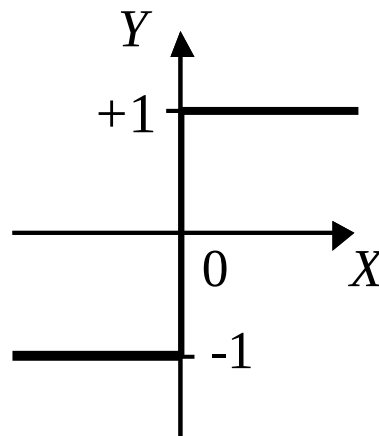
Activation functions of a neuron

Step function



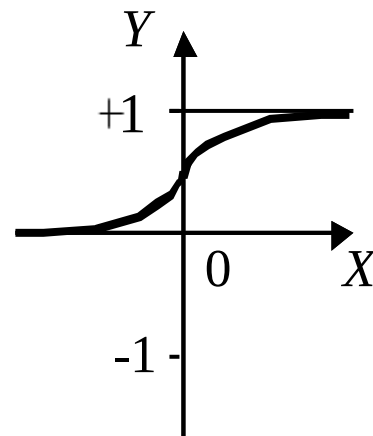
$$Y^{step} = \begin{cases} 1, & \text{if } X \geq 0 \\ 0, & \text{if } X < 0 \end{cases}$$

Sign function



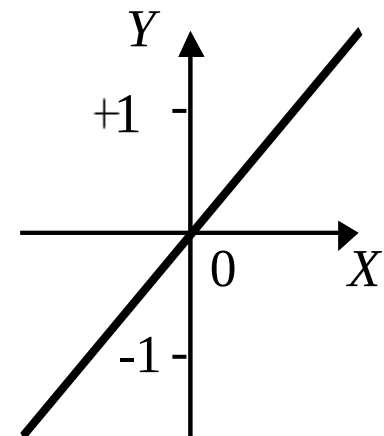
$$Y^{sign} = \begin{cases} +1, & \text{if } X \geq 0 \\ -1, & \text{if } X < 0 \end{cases}$$

Sigmoid function



$$Y^{sigmoid} = \frac{1}{1 + e^{-X}}$$

Linear function



$$Y^{linear} = X$$

Step Function

$$X = \sum_{i=1}^n x_i w_i$$

$$y = \begin{cases} 1 & \text{if } x \geq \theta \\ 0 & \text{if } x < \theta \end{cases}$$

Step $f(x) = 1$ if $x \geq \theta$, else 0

$X \equiv$ Net weighted inputs

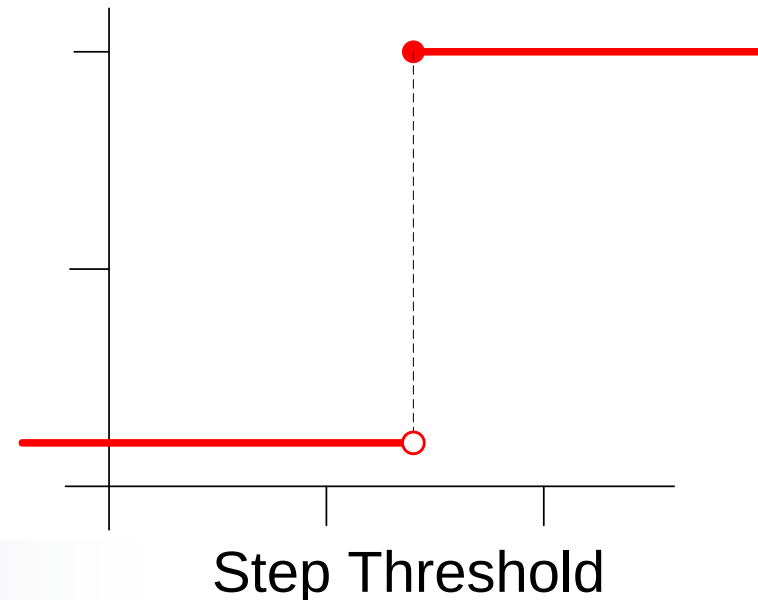
$x_i \equiv$ Inputs i

$w_i \equiv$ Weights i

$n \equiv$ Number of input neuron

$y \equiv$ The output of the neuron

$\theta \equiv$ Threshold



Sign Function

$$X = \sum_{i=1}^n x_i w_i$$

$$y = \begin{cases} +1 & \text{if } x \geq \theta \\ -1 & \text{if } x < \theta \end{cases}$$

$$y = \text{sign} \left[\sum_{i=1}^n x_i w_i - \theta \right]$$

$X \equiv$ Net weighted inputs

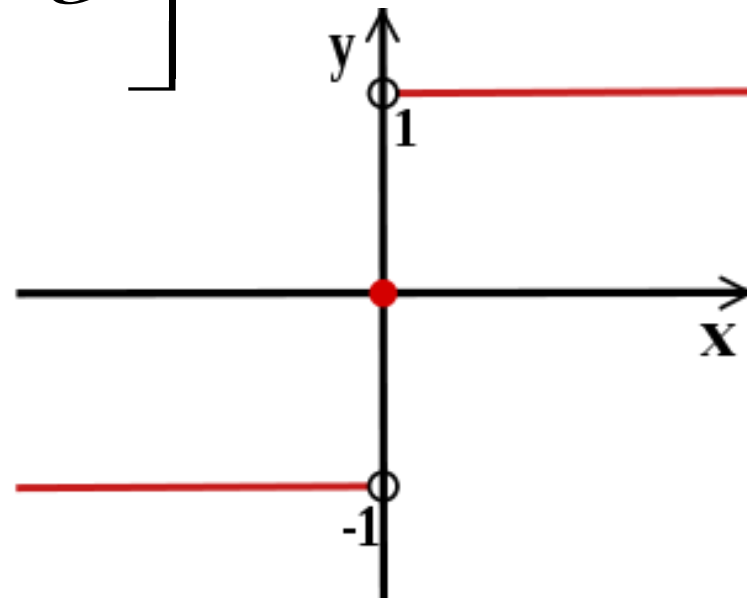
$x_i \equiv$ Inputs i

$w_i \equiv$ Weights i

$n \equiv$ Number of input neuron

$y \equiv$ The output of the neuron

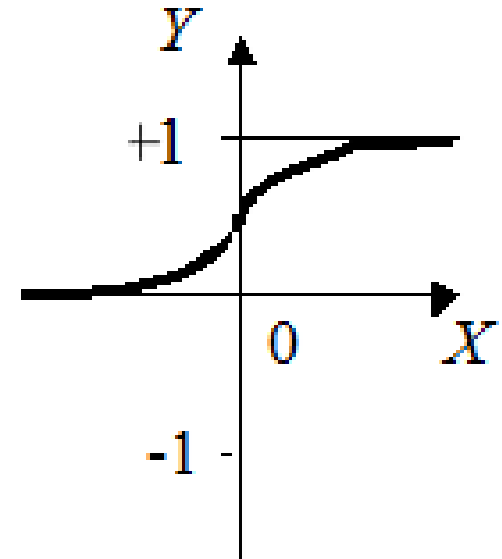
$\theta \equiv$ Threshold



Sigmoid Function

$$X = \sum_{i=1}^n x_i w_i$$

$$y = \frac{1}{1 + e^{-X}}$$



$X \equiv$ Net weighted inputs

$x_i \equiv$ Inputs i

$w_i \equiv$ Weights i

$n \equiv$ Number of input neuron

$y \equiv$ The output of the neuron

$\theta \equiv$ Threshold

Linear Function

$$X = \sum_{i=1}^n x_i w_i$$

$$y = x$$

$X \equiv$ Net weighted inputs

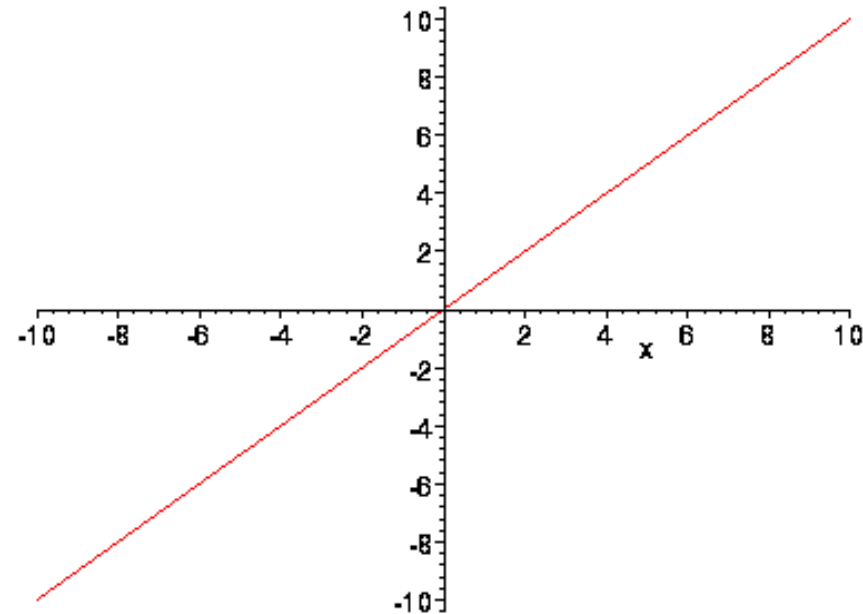
$x_i \equiv$ Inputs i

$w_i \equiv$ Weights i

$n \equiv$ Number of input neuron

$y \equiv$ The output of the neuron

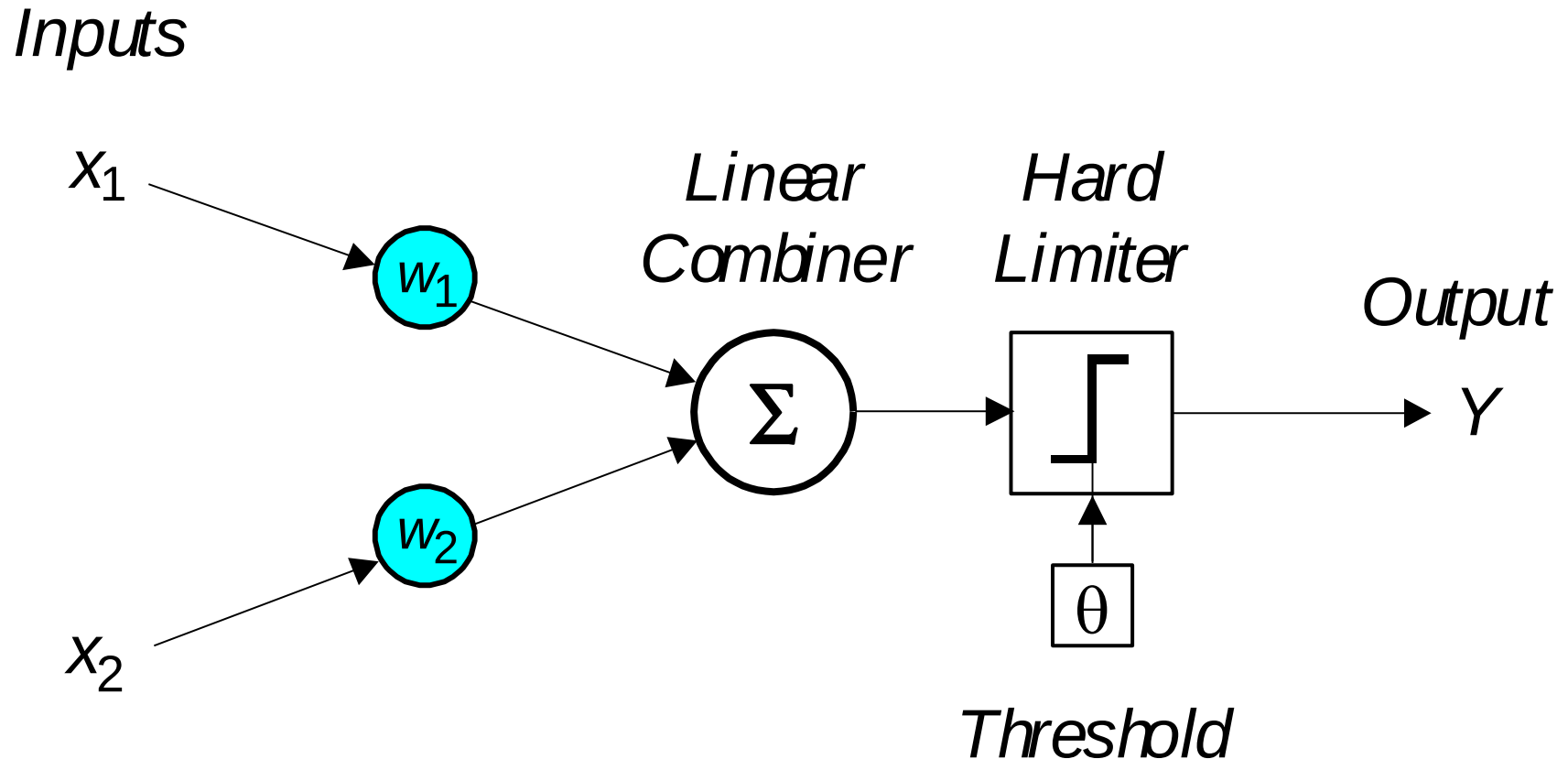
$\theta \equiv$ Threshold



Can a single neuron learn a task?

- In 1958, **Frank Rosenblatt** introduced a training algorithm that provided the first procedure for training a simple ANN: a **perceptron**.
- The perceptron is the simplest form of a neural network. It consists of a single neuron with *adjustable* synaptic weights and a *hard limiter*.

Single-layer two-input perceptron



The Perceptron

- The operation of **Rosenblatt's perceptron** is based on the **McCulloch and Pitts neuron model**. The model consists of a linear combiner followed by a hard limiter.
- The weighted sum of the inputs is applied to the hard limiter, which produces an output equal to +1 if its input is **positive** and -1 if it is **negative**.

<http://wwwold.ece.utep.edu/research/webfuzzy/docs/kk-thesis/kk-thesis-html/node12.html>

- The aim of the perceptron is to classify inputs, $x_1, x_2, \dots, x_n$, into one of two classes, say A_1 and A_2 .
- In the case of an elementary perceptron, the n -dimensional space is divided by a *hyperplane* into two decision regions.

How does the perceptron learn its classification tasks?

This is done by making small adjustments in the weights to reduce the difference between the **actual** and **desired** outputs of the perceptron. The initial weights are **randomly** assigned, usually in the range $[-0.5, 0.5]$, and then **updated** to obtain the output consistent with the training examples.

- If at **iteration p** , the actual output is $Y(p)$ and the desired output is $Y_d(p)$, then the error is given by:

$$e(p) = Y_d(p) - Y(p) \quad \text{where } p = 1, 2, 3, \dots$$

Iteration p here refers to the p th training example presented to the perceptron.

The perceptron learning rule

$$w_i(p+1) = w_i(p) + \alpha \cdot x_i(p) \cdot e(p)$$

where $p = 1, 2, 3, \dots$

α is the **learning rate**, a positive constant less than unity.

The perceptron learning rule was first proposed by **Rosenblatt** in 1960. Using this rule we can derive the perceptron training algorithm for classification tasks.

Perceptron's training algorithm

Step 1: Initialisation

Set initial weights $w_1, w_2, \dots, w_n$ and threshold θ to random numbers in the range $[-0.5, 0.5]$.

If the error, $e(p)$, is positive, we need to increase perceptron output $Y(p)$, but if it is negative, we need to decrease $Y(p)$.

Perceptron's training algorithm (continued)

Step 2: Activation

Activate the perceptron by applying inputs $x_1(p)$, $x_2(p), \dots, x_n(p)$ and desired output $Y_d(p)$.

Calculate the actual output at iteration $p = 1$

$$Y(p) = \text{step} \left[\sum_{i=1}^n x_i(p) w_i(p) - \theta \right]$$

where n is the number of the perceptron inputs, and step is a step activation function.

Perceptron's training algorithm (continued)

Step 3: Weight training

Update the weights of the perceptron

$$w_i(p+1) = w_i(p) + \Delta w_i(p)$$

where $\Delta w_i(p)$ is the weight correction at iteration p .

The weight correction is computed by the **delta rule**:

$$\Delta w_i(p) = \alpha \cdot x_i(p) \cdot e(p)$$

Step 4: Iteration

Increase iteration p by one, go back to *Step 2* and repeat the process until convergence.

Example of perceptron learning: the logical operation

Logical operation AND

Input variables		AND
x1	x2	X1 and x2
0	0	0
0	1	0
1	0	0
1	1	1

Threshold : $\theta=0.2$; learning rate : $\alpha=0.1$

EPOCH	Inputs		Desired output	Initial weights		Actual output	error	Final weights	
	x1	x2	yd	w1	w2	y	e	w1	w2
1	0	0	0	0.3	-0.1	0	0	0.3	-0.1
	0	1	0	0.3	-.01	0	0	0.3	-0.1
	1	0	0	0.3	-0.1	1	-1	0.2	-0.1
	1	1	1	0.2	-0.1	0	1	0.3	0.0
2	0	0	0	0.3	0.0	0	0	0.3	0.0
	0	1	0	0.3	0.0	0	0	0.3	0.0
	1	0	0	0.3	0.0	1	-1	0.2	0.0
	1	1	1	0.2	0.0	1	0	0.2	0.0
3	0	0	0	0.2	0.0	0	0	0.2	0.0
	0	1	0	0.2	0.0	0	0	0.2	0.0
	1	0	0	0.2	0.0	1	-1	0.1	0.0
	1	1	1	0.1	0.0	0	1	0.2	0.1
4	0	0	0	0.2	0.1	0	0	0.2	0.1
	0	1	0	0.2	0.1	0	0	0.2	0.1
	1	0	0	0.2	0.1	1	-1	0.1	0.1
	1	1	1	0.1	0.1	1	0	0.1	0.1

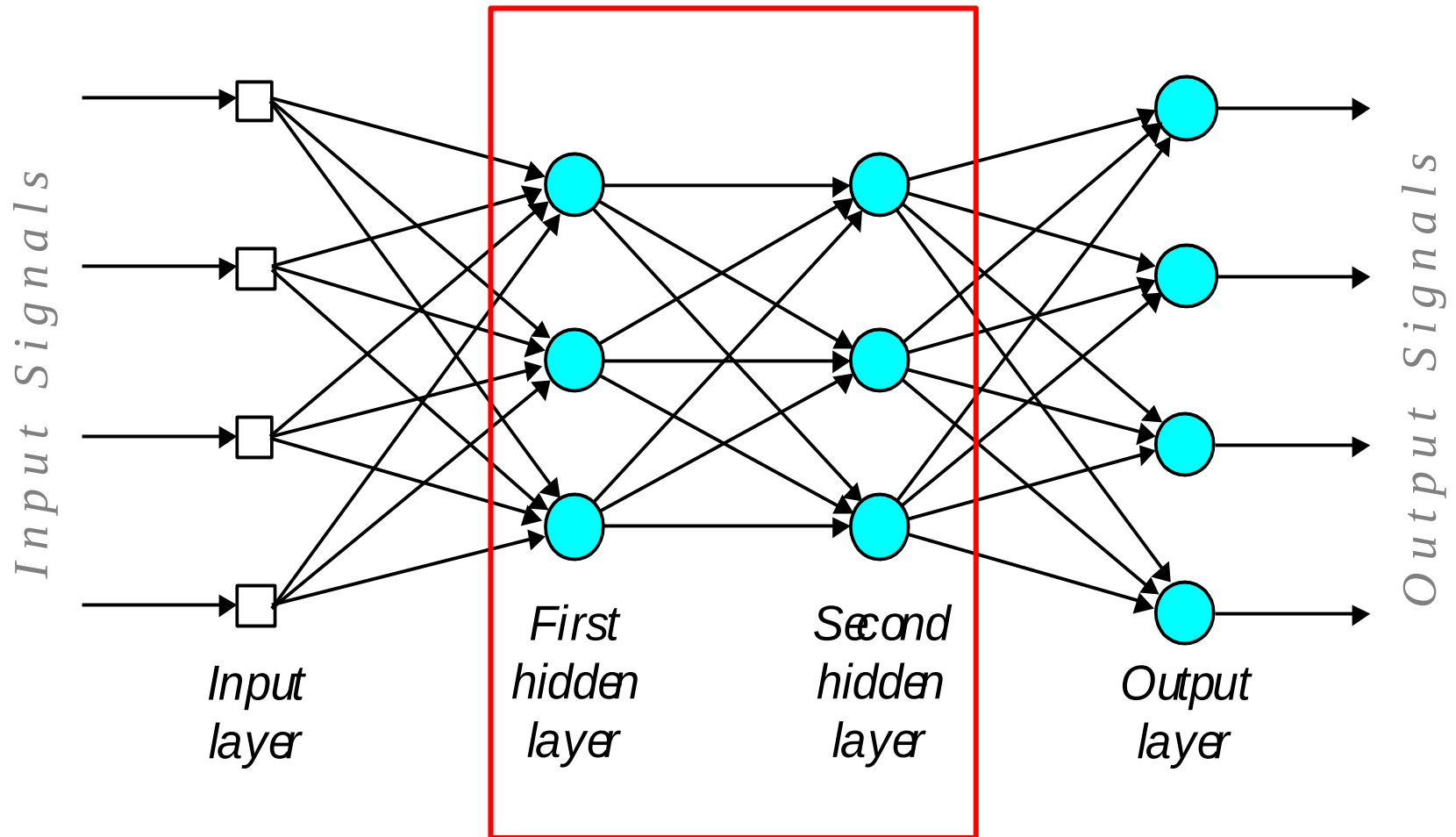
EPOCH	Inputs		Desire d output	Initial weights		Actual output	error	Final weights	
	x1	x2	yd	w1	w2	y	e	w1	w2
5	0	0	0	0.1	0.1	0	0	0.1	0.1
	0	1	0	0.1	0.1	0	0	0.1	0.1
	1	0	0	0.1	0.1	0	0	0.1	0.1
	1	1	1	0.1	0.1	1	0	0.1	0.1

Threshold : $\theta=0.2$; learning rate : $\alpha=0.1$

Multilayer neural networks

- A multilayer perceptron is a feedforward neural network with one or more hidden layers.
- The network consists of an **input layer** of source neurons, at least one **middle** or **hidden layer** of computational neurons, and an **output layer** of computational neurons.
- The input signals are propagated in a forward direction on **a layer-by-layer** basis.

Multilayer perceptron with two hidden layers



What does the middle layer hide?

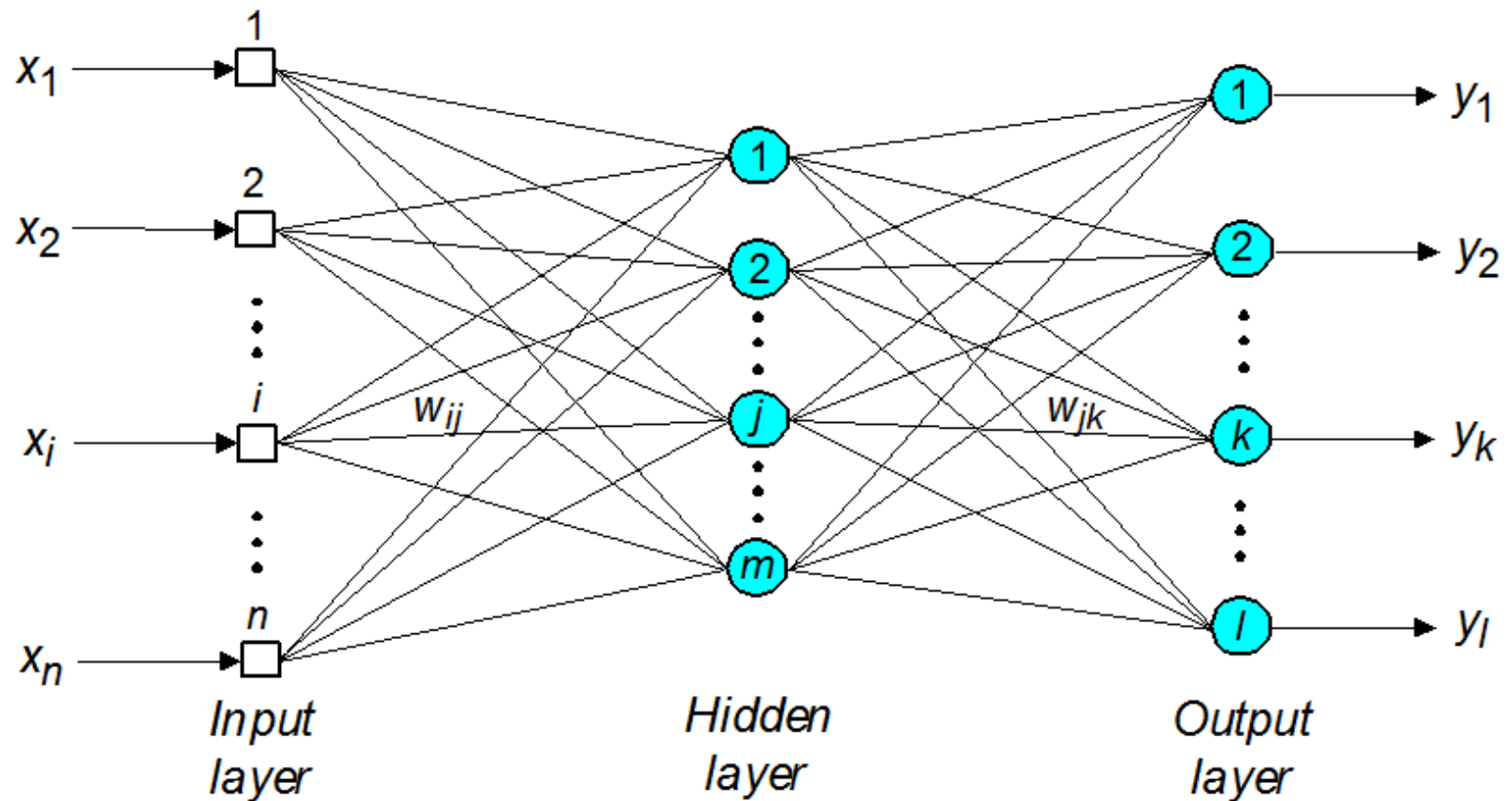
- A hidden layer “hides” its desired output. Neurons in the hidden layer cannot be observed through the input/output behaviour of the network. There is no obvious way to know what the desired output of the hidden layer should be.
- Commercial ANNs incorporate three and sometimes four layers, including **one or two hidden layers**. Each layer can contain from 10 to 1000 neurons. Experimental neural networks may have five or even six layers, including three or four **hidden layers**, and **utilise millions of neurons**.

Back-propagation neural network (BPNN)

- Learning in a multilayer network proceeds the same way as for a perceptron.
- A training set of input patterns is presented to the network.
- The network computes its output pattern, and if there is **an error** (the difference between **actual** and **desired** output patterns), the **weights** are adjusted to reduce this error.

- In a BPNN, the learning algorithm has two phases:
- First, a training input pattern is presented to the network input layer. The network propagates the input pattern from layer to layer until the output pattern is generated by the output layer.
- If this pattern is different from the desired output, an error is calculated and then propagated backwards through the network from the output layer to the input layer. The weights are modified as the error is propagated.

Three-layer back-propagation neural network



The back-propagation training algorithm

Step 1: Initialisation

Set all the weights and threshold levels of the network to random numbers uniformly distributed inside a small range:

$$\left(-\frac{2.4}{F_i}, +\frac{2.4}{F_i} \right)$$

where F_i is the total number of inputs of neuron i in the network. The weight initialisation is done on a neuron-by-neuron basis.

Step 2: Activation

Activate the BPNN by applying inputs $x_1(p), x_2(p), \dots, x_n(p)$ and desired outputs $y_{d,1}(p), y_{d,2}(p), \dots, y_{d,n}(p)$.

(a) Calculate the actual outputs of the neurons in the hidden layer:

$$y_j(p) = \text{sigmoid} \left[\sum_{i=1}^n x_i(p) \cdot w_{ij}(p) - \theta_j \right]$$

where n is the number of inputs of neuron j in the hidden layer, and *sigmoid* is the *sigmoid* activation function.

Step 2 : Activation (continued)

(b) Calculate the **actual outputs** of the neurons in the output layer:

$$y_k(p) = \text{sigmoid} \left[\sum_{j=1}^m x_{jk}(p) \cdot w_{jk}(p) - \theta_k \right]$$

where ***m*** is the number of inputs of neuron ***k*** in the output layer.

Step 3: Weight training

Update the weights in the BPNN propagating **backward** the errors associated with output neurons.

(a) Calculate the **error gradient** for the neurons in **the output layer**:

$$\delta_k(p) = y_k(p) \cdot [1 - y_k(p)] \cdot e_k(p)$$

where, $e_k(p) = y_{d,k}(p) - y_k(p)$

Calculate the weight corrections:

$$\Delta w_{jk}(p) = \alpha \cdot y_j(p) \cdot \delta_k(p)$$

Update the weights at the output neurons:

$$w_{jk}(p+1) = w_{jk}(p) + \Delta w_{jk}(p)$$

Step 3: Weight training (continued)

(b) Calculate the error gradient for the neurons in the hidden layer:

$$\delta_j(p) = y_j(p) \cdot [1 - y_j(p)] \cdot \sum_{k=1}^l \delta_k(p) w_{jk}(p)$$

Calculate the weight corrections:

$$\Delta w_{ij}(p) = \alpha \cdot x_i(p) \cdot \delta_j(p)$$

Update the weights at the hidden neurons:

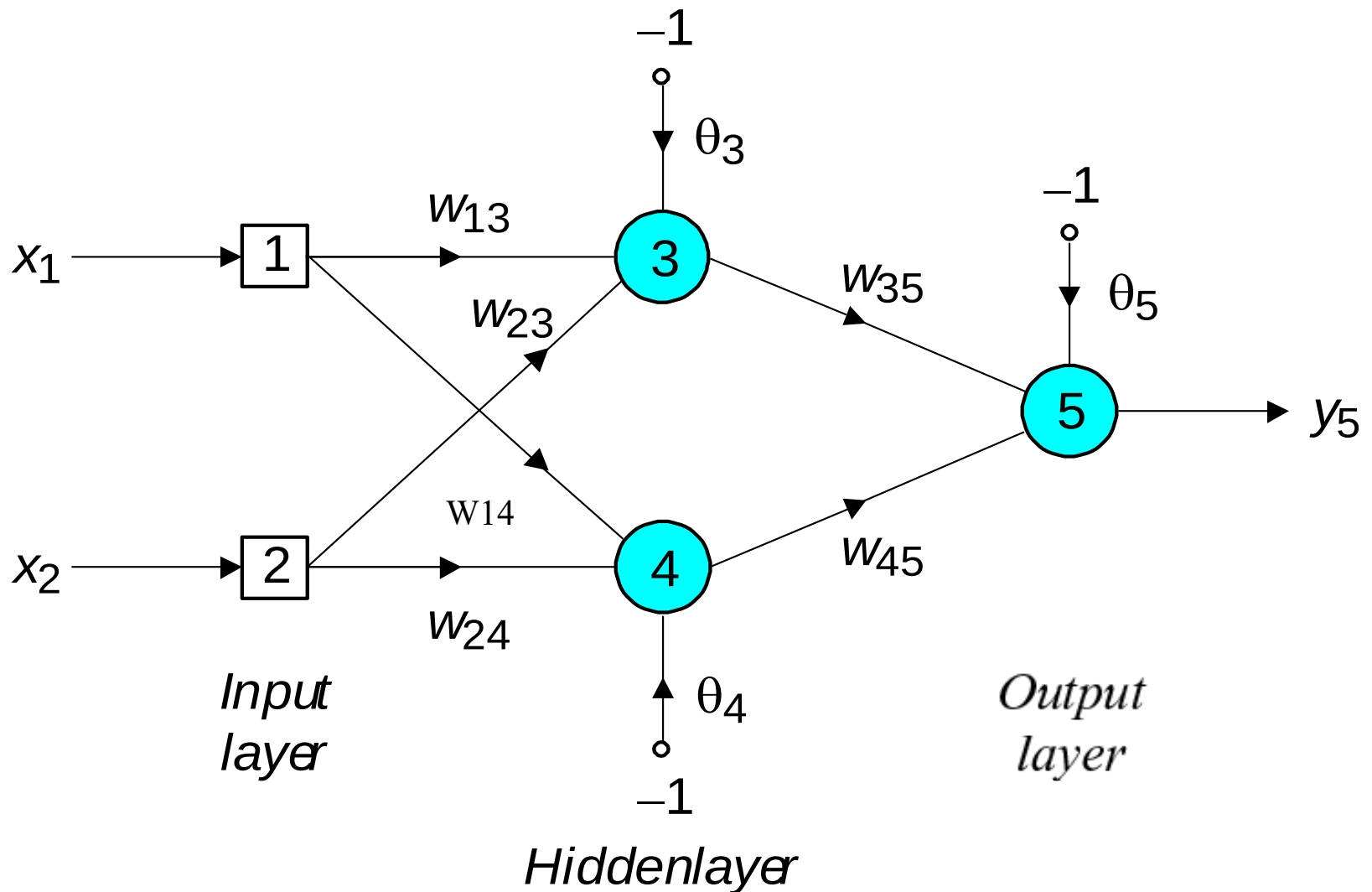
$$w_{ij}(p+1) = w_{ij}(p) + \Delta w_{ij}(p)$$

Step 4: Iteration

Increase iteration p by one, go back to Step 2 and **repeat** the process until the selected error criterion is satisfied.

As an example, we may consider the three-layer back-propagation network. Suppose that the network is required to perform logical operation ***Exclusive-OR***. Recall that a single-layer perceptron could not do this operation. Now we will apply the three-layer net.

Three-layer network for solving the Exclusive-OR operation



- The effect of the threshold applied to a neuron in the hidden or output layer is represented by its weight, θ , connected to a fixed input equal to -1 .
- The initial weights and threshold levels are set randomly as follows:

$$w_{13} = 0.5, w_{14} = 0.9, w_{23} = 0.4, w_{24} = 1.0, w_{35} = -1.2,$$

$$w_{45} = 1.1, \theta_3 = 0.8, \theta_4 = -0.1 \text{ and } \theta_5 = 0.3.$$



- We consider a training set where inputs x_1 and x_2 are equal to 1 and desired output $y_{d,5}$ is 0. The actual outputs of neurons 3 and 4 in the hidden layer are calculated as

$$y_3 = \text{sigmoid}(x_1 w_{13} + x_2 w_{23} - \theta_3) = 1 / \left[1 + e^{-(1 \cdot 0.5 + 1 \cdot 0.4 - 1 \cdot 0.8)} \right] = 0.5250$$

$$y_4 = \text{sigmoid}(x_1 w_{14} + x_2 w_{24} - \theta_4) = 1 / \left[1 + e^{-(1 \cdot 0.9 + 1 \cdot 1.0 + 1 \cdot 0.1)} \right] = 0.8808$$

- Now the actual output of neuron 5 in the output layer is determined as:

$$y_5 = \text{sigmoid}(y_3 w_{35} + y_4 w_{45} - \theta_5) = 1 / \left[1 + e^{-(-0.5250 \cdot 1.2 + 0.8808 \cdot 1.1 - 1 \cdot 0.3)} \right] = 0.5097$$

- Thus, the following error is obtained:

$$e = y_{d,5} - y_5 = 0 - 0.5097 = -0.5097$$

- The next step is weight training. To update the weights and threshold levels in our network, we propagate the error, e , from the output layer backward to the input layer.
- First, we calculate the error gradient for neuron 5 in the output layer:

$$\delta_5 = y_5 (1 - y_5) e = 0.5097 \cdot (1 - 0.5097) \cdot (-0.5097) = -0.1274$$

- Then we determine the weight corrections assuming that the learning rate parameter, α , is equal to 0.1:

$$\Delta w_{35} = \alpha \cdot y_3 \cdot \delta_5 = 0.1 \cdot 0.5250 \cdot (-0.1274) = -0.0067$$

$$\Delta w_{45} = \alpha \cdot y_4 \cdot \delta_5 = 0.1 \cdot 0.8808 \cdot (-0.1274) = -0.0112$$

$$\Delta \theta_5 = \alpha \cdot (-1) \cdot \delta_5 = 0.1 \cdot (-1) \cdot (-0.1274) = -0.0127$$

- Next we calculate the error gradients for neurons 3 and 4 in the hidden layer:

$$\delta_3 = y_3(1 - y_3) \cdot \delta_5 \cdot w_{35} = 0.5250 \cdot (1 - 0.5250) \cdot (-0.1274) \cdot (-1.2) = 0.0381$$

$$\delta_4 = y_4(1 - y_4) \cdot \delta_5 \cdot w_{45} = 0.8808 \cdot (1 - 0.8808) \cdot (-0.1274) \cdot 1.1 = -0.0147$$

- We then determine the weight corrections:

$$\Delta w_{13} = \alpha \cdot x_1 \cdot \delta_3 = 0.1 \cdot 1 \cdot 0.0381 = 0.0038$$

$$\Delta w_{23} = \alpha \cdot x_2 \cdot \delta_3 = 0.1 \cdot 1 \cdot 0.0381 = 0.0038$$

$$\Delta \theta_3 = \alpha \cdot (-1) \cdot \delta_3 = 0.1 \cdot (-1) \cdot 0.0381 = -0.0038$$

$$\Delta w_{14} = \alpha \cdot x_1 \cdot \delta_4 = 0.1 \cdot 1 \cdot (-0.0147) = -0.0015$$

$$\Delta w_{24} = \alpha \cdot x_2 \cdot \delta_4 = 0.1 \cdot 1 \cdot (-0.0147) = -0.0015$$

$$\Delta \theta_4 = \alpha \cdot (-1) \cdot \delta_4 = 0.1 \cdot (-1) \cdot (-0.0147) = 0.0015$$

■ At last, we update all weights and threshold:

$$w_{13} = w_{13} + \Delta w_{13} = 0.5 + 0.0038 = 0.5038$$

$$w_{14} = w_{14} + \Delta w_{14} = 0.9 - 0.0015 = 0.8985$$

$$w_{23} = w_{23} + \Delta w_{23} = 0.4 + 0.0038 = 0.4038$$

$$w_{24} = w_{24} + \Delta w_{24} = 1.0 - 0.0015 = 0.9985$$

$$w_{35} = w_{35} + \Delta w_{35} = -1.2 - 0.0067 = -1.2067$$

$$w_{45} = w_{45} + \Delta w_{45} = 1.1 - 0.0112 = 1.0888$$

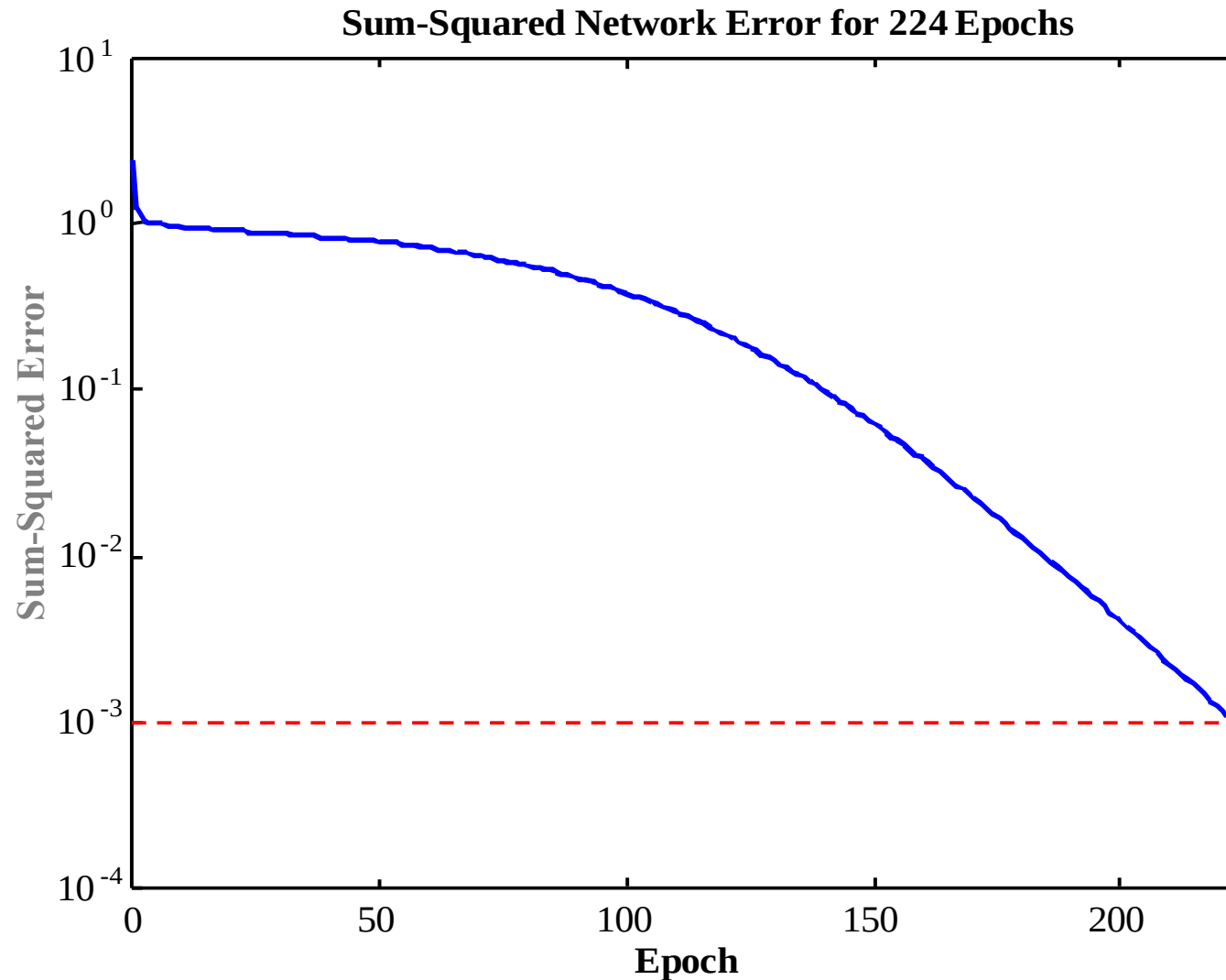
$$\theta_3 = \theta_3 + \Delta \theta_3 = 0.8 - 0.0038 = 0.7962$$

$$\theta_4 = \theta_4 + \Delta \theta_4 = -0.1 + 0.0015 = -0.0985$$

$$\theta_5 = \theta_5 + \Delta \theta_5 = 0.3 + 0.0127 = 0.3127$$

■ The training process is repeated until the sum of squared errors is less than 0.001.

Learning curve for operation *Exclusive-OR*



Final results of three-layer network learning

Inputs		Desired output	Actual output	Error	Sum of squared errors
x_1	x_2	y_d	y_5	e	
1	1	0	0.0155	-0.0155	0.0010
0	1	1	0.9849	0.0151	
1	0	1	0.9849	0.0151	
0	0	0	0.0175	-0.0175	

END